PROJECT DOCUMENTATION

StockPulse – Real-Time Investment Tracking Dashboard

Subject: React JS
Project No.: 195
Program: B.Tech CSE (2025-29)
University: ITM Skills University
Semester: Semester II

Submitted By: Biswajeet Rout
Roll Number: 150096725120
Cohort: Jeff Bezos

Submission: Live Deployment | GitHub Repository

3.1 Abstract

StockPulse is a frontend-only React 19 application that delivers a real-time investment tracking dashboard with simulated live stock price updates, portfolio management, trade execution, and transaction history visualization — all in a single, performant browser-based interface. The project is designed to demonstrate modern React patterns, performance optimization techniques, and developer-grade UI composition using the latest React ecosystem tooling.

The application runs entirely on the client side with no backend server or external API dependency, relying instead on simulated market data generated at runtime. It features a real-time price ticker with flash animations, a drag-and-drop watchlist organizer with mini sparkline charts, a trade execution engine with configurable limits and balance validation, virtualized scrolling for 1000+ transaction records, and an investor account summary with portfolio valuation and net worth calculation. An error boundary component ensures graceful recovery from rendering errors in financial calculations.

3.2 Student Details

Field	Detail
Student Name	Biswajeet Rout
Roll Number	150096725120
Cohort	Jeff Bezos
Program	B.Tech CSE (2025-29)
University	ITM Skills University
Subject	React JS
Semester	Semester II
Project Number	195
Project Title	StockPulse – Real-Time Investment Tracking Dashboard
Implementation	React 19, Vite, @dnd-kit, Recharts (frontend-only)

3.3 Problem Statement

Retail investors and finance enthusiasts who want to track a personal portfolio in real time currently face several practical limitations with generic tools and spreadsheets:

Problem	Description
No real-time view	Spreadsheets and static dashboards do not reflect live price movements; manual refresh is re
Scattered information	Portfolio value, holdings, and recent trades are spread across multiple apps or tabs.
No watchlist prioritization	Users cannot reorder tracked stocks to reflect current focus without editing underlying data.
Performance at scale	Rendering hundreds of transaction rows in a standard list causes visible UI lag.
No trade simulation	Tools rarely let users simulate buy/sell operations against a live balance with configurable lim

Problem	Description
Absent error handling	Financial calculation errors typically crash the entire UI rather than being caught gracefully.

StockPulse addresses each of these problems with a dedicated, purpose-built React interface implemented entirely in the browser, without a server, database, or external API dependency.

3.4 Objectives

- 1. Render a real-time price ticker that updates simulated stock prices every 800 ms with green/red flash animations to signal price direction.
- 2. Implement a custom watchlist organizer with drag-and-drop reordering using @dnd-kit, each stock card showing a mini Recharts sparkline of recent price history.
- 3. Build a trade execution panel that validates buy/sell orders against configurable per-trade limits and the investor's available balance, rejecting invalid trades with clear feedback.
- 4. Provide a lag-free transaction history view using virtual scrolling — rendering only visible rows plus a buffer — to handle 1000+ records without UI degradation.
- 5. Display an investor account summary showing total portfolio valuation, net worth, and account profile details, updated reactively as trades execute.
- 6. Implement a calculation error boundary that catches rendering errors inside financial components, displays a descriptive fallback UI, and prevents a single error from crashing the whole app.
- 7. Optimize rendering with React.memo on transaction rows, useMemo for portfolio calculations and chart data, and useCallback for stable scroll handler references.
- 8. Use a SmartWrapper component to isolate ticker item re-renders, preventing cascading updates from propagating through unrelated parts of the component tree.
- 9. Implement dynamic container height measurement via ResizeObserver so the virtual scroll viewport adapts correctly to layout changes.
- 10. Deliver the complete application via Vite for fast local development and optimized production builds.

3.5 Design / Architecture

System Overview

StockPulse follows a tab-based single-page architecture with five dedicated views, each implemented as an isolated React component subtree. A shared state layer manages the portfolio, watchlist, trade history, and simulated price feed, passing data down via props and context to avoid prop-drilling in deeply nested components.

Application Tabs

Tab	Responsibility
Overview	Real-time price ticker for all tracked stocks with 800 ms update interval and price flash animations.
Watchlist	Drag-and-drop reorderable list of favourite stocks, each with a mini Recharts sparkline of recent pric
Trade	Buy/sell execution panel with trade limit configuration, balance display, and validation feedback.
History	Virtually scrolled transaction log rendering 1000+ records; only visible rows are mounted in the DOM
Account	Investor profile card showing portfolio valuation, net worth breakdown, and account details.

Module Mapping

Module	Responsibility	Implementation
Price Ticker	Simulates live price updates every 800 ms	useEffect + setInterval + SmartWrapper

Module	Responsibility	Implementation
Watchlist Organizer	Drag-and-drop ordering + sparkline charts	@dnd-kit/sortable + Recharts LineChart
Trade Engine	Executes buy/sell with limit and balance checks	useState + validation logic
Virtual Scroller	Renders only visible transaction rows + buffer	Custom hook + ResizeObserver
Account Panel	Portfolio value and net worth calculation	useMemo + derived state
Error Boundary	Catches rendering errors in financial calculations	React Component + componentDidCatch
SmartWrapper	Isolates per-ticker re-renders from parent list	React.memo wrapper component

3.6 Technical Implementation

3.6.1 Simulated Real-Time Price Feed

A single useEffect hook at the top of the Overview tab sets up a setInterval that fires every 800 ms. On each tick, all tracked stock prices are updated by a random delta within a configurable percentage range, and a flash class (green for increase, red for decrease) is applied to the changed price cell. The animation resets after 400 ms via a follow-up setState call, giving a clean two-frame flash effect without external animation libraries.

3.6.2 SmartWrapper Re-Render Isolation

Each ticker row is wrapped in a SmartWrapper component that applies React.memo with a custom comparison function. The comparison checks only the stock's price and flash state, ignoring unrelated props. This means a price change in AAPL does not trigger a re-render in the TSLA row, keeping the ticker list performant even with frequent updates across many stocks.

3.6.3 Drag-and-Drop Watchlist

The watchlist uses @dnd-kit/sortable to implement pointer and touch-friendly drag-and-drop reordering. Each watchlist item is wrapped in a useSortable hook which exposes transform and transition CSS values applied as inline styles, providing smooth visual feedback during drag. On drop, the items array is reordered using the arrayMove utility and saved to local state, persisting the new order for the session.

3.6.4 Virtual Scrolling for Transaction History

The transaction history renders 1000+ records using a custom virtual scrolling implementation. A ResizeObserver watches the scroll container's height; a useCallback-stabilized scroll handler calculates which items fall within the visible window plus a buffer of 5 rows on each side. Only those rows are mounted in the DOM, with the remaining height held by a spacer div. This keeps DOM node count constant regardless of total transaction volume.

3.6.5 Trade Execution with Limit Validation

The trade panel maintains a configurable max-trade-limit value alongside the investor's current cash balance in React state. On submit, the execution handler validates that: (a) the order value does not exceed the per-trade limit, (b) for buys, the investor has sufficient balance, and (c) for sells, the investor holds enough shares. Failing any check surfaces an inline error message without executing the trade. Successful trades update the portfolio state and append a new record to the transaction log.

3.6.6 Portfolio Calculation with useMemo

Portfolio valuation and net worth are derived values computed via useMemo. The memoized selector recomputes only when the holdings array or the current price map changes, avoiding expensive recalculation on unrelated state updates such as the ticker flash state toggling every 400 ms.

3.6.7 Error Boundary for Financial Components

A class-based ErrorBoundary component wraps the portfolio valuation and trade summary sections. It overrides componentDidCatch to log the error and getDerivedStateFromError to set a hasError flag. When hasError is true, the boundary renders a descriptive fallback UI instead of the broken subtree, allowing the rest of the dashboard to continue functioning.

3.7 Technical Specifications

Metric	Value
React version	React 19
Build tool	Vite
Drag-and-drop library	@dnd-kit/sortable
Charting library	Recharts
Price update interval	800 ms
Flash animation duration	400 ms
Virtual scroll buffer	5 rows per side
Simulated transaction records	1000+
Client-side processing	100%
External runtime dependencies	0 (no backend, no live API)
Development server port	localhost:5173
Production preview port	localhost:4173
React Hooks used	useState, useEffect, useMemo, useCallback, useRef
Key optimization techniques	React.memo, SmartWrapper, Virtual Scrolling, ResizeObserver

3.8 Screenshots of Execution

The following screenshots demonstrate StockPulse running on the development server. Each screenshot corresponds to one tab of the dashboard.

[Screenshot placeholder — Fig 3.1 — Overview Tab: Real-time price ticker with green/red flash animations on live price updates.]

Fig 3.1 — Overview Tab: Real-time price ticker with green/red flash animations on live price updates.

[Screenshot placeholder — Fig 3.2 — Watchlist Tab: Drag-and-drop reorderable stock cards with mini Recharts sparkline price charts.]

Fig 3.2 — Watchlist Tab: Drag-and-drop reorderable stock cards with mini Recharts sparkline price charts.

[Screenshot placeholder — Fig 3.3 — Trade Tab: Buy/sell execution panel with trade limit configuration and balance validation.]

Fig 3.3 — Trade Tab: Buy/sell execution panel with trade limit configuration and balance validation.

[Screenshot placeholder — Fig 3.4 — History Tab: Virtually scrolled transaction log rendering 1000+ records with smooth performance.]

Fig 3.4 — History Tab: Virtually scrolled transaction log rendering 1000+ records with smooth performance.

[Screenshot placeholder — Fig 3.5 — Account Tab: Investor profile showing portfolio valuation, net worth, and account details.]

Fig 3.5 — Account Tab: Investor profile showing portfolio valuation, net worth, and account details.

3.9 Results and Findings

Real-Time Price Ticker

The ticker updates all tracked stock prices every 800 ms without perceptible lag. SmartWrapper re-render isolation ensures only rows whose prices changed are re-rendered on each tick, keeping the browser frame budget well within 16 ms even with 15+ stocks displayed.

Watchlist Drag-and-Drop

Drag-and-drop reordering works correctly on both pointer and touch inputs. Items animate smoothly during drag via CSS transform transitions applied by @dnd-kit, and the new order is preserved in state after drop. Each sparkline reflects the last N price samples for that stock, updating live as new prices arrive.

Trade Execution and Validation

All three validation paths (limit exceeded, insufficient balance for buy, insufficient shares for sell) surface clear inline error messages without submitting the trade. Successful trades deduct from the cash balance, add shares to the portfolio, and append a timestamped record to the transaction log immediately.

Virtual Scrolling Performance

The transaction history renders 1000 pre-seeded records with no measurable scroll jank. Profiling confirms that only approximately 20–30 rows are present in the DOM at any time, with the ResizeObserver correctly recalculating the visible window on container resize.

Portfolio Calculation

useMemo correctly prevents recalculation of portfolio valuation during ticker flash state toggles, which would otherwise trigger expensive recomputation every 400 ms. Valuation updates are triggered only on actual price or holdings changes, as confirmed by React DevTools profiler.

Error Boundary

Injecting a deliberate rendering error into the portfolio calculation component confirms that the error boundary catches the exception, renders the fallback UI in place of the broken section, and leaves all other dashboard tabs fully functional.

3.10 Limitations

- Stock price data is entirely simulated at runtime using random deltas. No live market data API is integrated; prices do not reflect actual market values.
- Portfolio state is held in React component state and is lost on page refresh. No LocalStorage or backend persistence is implemented for holdings or transaction history.
- The trade execution engine does not model partial fills, order books, or spread, and cannot represent real brokerage constraints.
- The virtual scrolling implementation uses a fixed estimated row height for offset calculation. Variable-height rows would require a more complex measurement strategy.
- Drag-and-drop watchlist order is not persisted across page reloads; reordering is session-only.
- The error boundary covers the financial calculation subtree only; unhandled errors in other parts of the component tree would still propagate to the React root.

3.11 Conclusion

StockPulse successfully delivers a performant, feature-rich investment tracking dashboard built entirely in React 19, demonstrating a range of modern React patterns and performance optimization techniques in a realistic domain. The project meets each objective: real-time price updates with animation, drag-and-drop watchlist management, validated trade execution, lag-free virtual scrolling for large transaction histories, portfolio valuation, and graceful error handling — all without a server or external data dependency.

The implementation demonstrates practical application of React.memo, useMemo, useCallback, ResizeObserver, and class-based error boundaries, and shows how targeted re-render isolation via SmartWrapper can maintain smooth UI performance even under frequent state updates from a simulated live data feed.

Future enhancements could include LocalStorage persistence for portfolio state and watchlist order, integration with a live market data API such as Finnhub or Alpha Vantage, server-side trade logging via a Node.js/Express backend, and a news feed panel correlating headlines with tracked tickers.

Submitted by: Biswajeet Rout | Roll: 150096725120 | Cohort: Jeff Bezos

ITM Skills University | B.Tech CSE | Semester II